

B2-1 가계부 콘솔 프로그램

평가자 질문사항에 대한 답변

질문마다 요약 답변 · 코드 근거(파일:행 인용) · 실행 근거(명령어와 실제 출력)
항목 1 기능·영속성 항목 2 구조·책임 항목 3 파이썬 기법 항목 4 설계 판단

시연 준비 | 실행 환경과 순서

실행 근거 (b2-1 폴더에서 실행)

① 가져오기 테스트용 CSV 준비 (2번째 데이터 행은 일부러 깨진 행)

```
$ cd b2-1
$ cat > imp.csv <<'EOF'
date,type,category,amount,memo,tags
2024-01-20,expense,transport,1250,지하철,
2024-01-21,expense,없는카테고리,1000,깨짐,
2024-01-22,income,salary,50000,보너스,
EOF
$ cat imp.csv          # 헤더 + 3행 확인
```

② 모든 명령은 빈 demo 폴더를 대상으로 실행

```
$ python3 -m budget_app <명령> [옵션] --data-dir demo
```

읽는 법

- 이후 슬라이드의 명령을 순서대로 실행하면 같은 출력이 나옵니다.
- 거래 id(TX-000001...)는 빈 폴더 기준으로 매겨집니다.
- 코드 근거의 왼쪽 숫자는 실제 파일의 행 번호입니다.

주의: --data-dir는 명령 이름 뒤에

- 맞춤: python3 -m budget_app add --data-dir demo
 - 틀림: python3 -m budget_app --data-dir demo add
- 하위 명령의 기본값(data)이 앞의 값을 덮어써서 실제 data/ 폴더에 기록됩니다.

명령 복사는 PDF 대신 평가/시연_명령어.sh에서

- PDF는 긴 줄이 화면 폭에서 끊긴 채로 복사됩니다.
- 전체 실행: zsh 평가/시연_명령어.sh

시연 후 정리: rm -r demo imp.csv out.csv

항목 1-1 | 8개 명령 동작 (1/2): add · list · search

Q. add/list/search/summary/export/import/update/delete가 요구사항대로 동작하는가?

A. 8개 명령은 모두 argparse 하위 명령으로 등록되고, main()이 각 cmd_* 함수로 연결합니다.
add는 입력을 검증해 파일 끝에 한 줄 추가, list는 최신순, search는 조건에 맞는 거래만 출력합니다.

코드 근거 (budget_app/파일:행)

cli.py:232, 235, 239, 248, 269, 279, 283, 287

```
232     p_add = subparsers.add_parser("add", parents=[common], help="거래  
추가 (대화형)")  
:  
235     p_list = subparsers.add_parser("list", parents=[common], help="거  
래 목록 (최신순)")  
:  
239     p_search = subparsers.add_parser("search", parents=[common],  
help="조건 기반 거래 검색")  
:  
248     p_summary = subparsers.add_parser("summary", parents=[common],  
help="월별 요약")  
:  
269     p_update = subparsers.add_parser("update", parents=[common],  
help="거래 수정 (옵션 기반)")  
:  
279     p_delete = subparsers.add_parser("delete", parents=[common],  
help="거래 삭제")  
:  
283     p_import = subparsers.add_parser("import", parents=[common],  
help="CSV 가져오기")  
:  
287     p_export = subparsers.add_parser("export", parents=[common],  
help="CSV 내보내기")
```

실행 근거 (b2-1 폴더에서 실행)

```
$ printf '2024-01-15\nexpense\nfood\n15000\n점심\nmeal\n' | python3 -m  
budget_app add --data-dir demo  
... [저장 완료] id=TX-000001  
$ printf '2024-01-25\nincome\nsalary\n3000000\n월급\n\n' | python3 -m  
budget_app add --data-dir demo  
... [저장 완료] id=TX-000002
```

```
$ python3 -m budget_app list --data-dir demo  
TX-000002 | 2024-01-25 | income | salary | 3000000 | 월급  
TX-000001 | 2024-01-15 | expense | food | 15000 | 점심 | #meal
```

```
$ python3 -m budget_app search --category food --data-dir demo  
TX-000001 | 2024-01-15 | expense | food | 15000 | 점심 | #meal
```

코드 근거 (budget_app/파일:행)

cli.py:297-300

```
297 def main(argv: list[str] | None = None) -> int:  
298     parser = build_parser()  
299     args = parser.parse_args(argv)  
300     return args.func(args)
```

항목 1-1 | 8개 명령 동작 (2/2): summary · update · import · export · delete

Q. add/list/search/summary/export/import/update/delete가 요구사항대로 동작하는가?

A. summary는 월별 수입·지출·잔액·예산 사용률, update/delete는 id로 거래를 찾아 파일에 반영, import/export는 CSV로 거래를 주고받습니다. 모두 종료 코드 0으로 끝납니다.

코드 근거 (budget_app/파일:행)

service.py:230-238

```
230         for tx in self.tx_repo.iter_all():
231             if not tx.date.startswith(month):
232                 continue
233             found_any = True
234             if tx.type == "income":
235                 income += tx.amount
236             else:
237                 expense += tx.amount
238                 category_expense[tx.category] =
category_expense.get(tx.category, 0) + tx.amount
```

service.py:202-207

```
202         found = self.tx_repo.update(tx_id, apply_changes)
203         if not found:
204             raise AppError(
205                 f"존재하지 않는 거래 id입니다: {tx_id}",
206                 hint="`list` 명령으로 올바른 id를 확인하세요.",
207             )
```

service.py:210-211

```
210         def delete_transaction(self, tx_id: str) -> None:
211             if not self.tx_repo.delete(tx_id):
```

실행 근거 (b2-1 폴더에서 실행)

```
$ python3 -m budget_app budget set --month 2024-01 --amount 100000 --
data-dir demo
[저장 완료] 2024-01 예산 100000원
$ python3 -m budget_app summary --month 2024-01 --data-dir demo
총 수입: 3000000원
총 지출: 15000원
잔액: 2985000원
예산: 100000원 (사용률 15.0%)

지출 TOP 5
1) food 15000원
```

```
$ python3 -m budget_app update --id TX-000001 --amount 18000 --data-dir
demo
[수정 완료] id=TX-000001
$ python3 -m budget_app import --from imp.csv --data-dir demo
[완료] imported=2, skipped=1
$ python3 -m budget_app export --out out.csv --month 2024-01 --data-dir
demo
[완료] out.csv (4 records)
$ python3 -m budget_app delete --id TX-000002 --data-dir demo
[삭제 완료] id=TX-000002
```

항목 1-2 | 재실행 후 데이터 유지 (저장 파일 3개)

Q. 프로그램 재실행 후에도 거래/카테고리/예산 데이터가 유지되는가? (저장 파일 3개 이상)

A. 거래·카테고리·예산을 JSONL 파일 3개에 나누어 저장합니다. 명령마다 새 프로세스로 실행되므로, 다음 명령에서 앞 명령의 결과가 보이는 것 자체가 재실행 후 유지된다는 근거입니다.

코드 근거 (budget_app/파일:행)

cli.py:16-21

```
16 def _build_service(data_dir: str) -> BudgetService:
17     base = Path(data_dir)
18     tx_repo = TransactionRepository(base / "transactions.jsonl")
19     cat_store = CategoryStore(base / "categories.jsonl")
20     budget_store = BudgetStore(base / "budgets.jsonl")
21     return BudgetService(tx_repo, cat_store, budget_store)
```

repository.py:40-43

```
40 def _append_jsonl(path: Path, row: dict) -> None:
41     path.parent.mkdir(parents=True, exist_ok=True)
42     with path.open("a", encoding="utf-8") as f:
43         f.write(json.dumps(row, ensure_ascii=False) + "\n")
```

실행 근거 (b2-1 폴더에서 실행)

```
$ printf 'hobby\n' | python3 -m budget_app category add --data-dir demo
카테고리명: [저장 완료] category=hobby
$ ls demo
budgets.jsonl  categories.jsonl  transactions.jsonl
```

```
$ python3 -m budget_app list --data-dir demo
TX-000004 | 2024-01-22 | income | salary | 50000 | 보너스
TX-000003 | 2024-01-20 | expense | transport | 1250 | 지하철
TX-000001 | 2024-01-15 | expense | food | 18000 | 점심 | #meal
$ python3 -m budget_app category list --data-dir demo
- food
- transport
- rent
- salary
- etc
- hobby
$ cat demo/budgets.jsonl
{"month": "2024-01", "amount": 100000}
```

항목 1-3 | category add/list/remove와 사용 중 카테고리 보호

Q. category add/list/remove가 정상 동작하는가? (삭제 시 사용 중인 카테고리 처리 포함)

A. remove는 먼저 거래 파일에서 사용 중인 카테고리 집합을 구해, 사용 중이면 AppError로 거부합니다.
사용하지 않는 카테고리만 categories.jsonl에서 제거합니다. (add/list 실행은 앞 슬라이드 참고)

코드 근거 (budget_app/파일:행)

service.py:88-96

```
88     def remove_category(self, name: str) -> None:
89         if not self.cat_store.exists(name):
90             raise AppError(f"존재하지 않는 카테고리입니다: {name}")
91         if name in self.tx_repo.used_categories():
92             raise AppError(
93                 f"'{name}' 카테고리를 사용 중인 거래가 있어 삭제할 수 없습니다.",
94                 hint="해당 카테고리를 사용하는 거래를 먼저 다른 카테고리로 옮기거나
삭제하세요.",
95             )
96         self.cat_store.remove(name)
```

repository.py:118-119

```
118     def used_categories(self) -> set[str]:
119         return {tx.category for tx in self.iter_all()}
```

repository.py:150-151

```
150         names.remove(name)
151         _atomic_write_jsonl(self.path, [{"name": n} for n in names])
```

실행 근거 (b2-1 폴더에서 실행)

사용 중인 food → 거부, 종료 코드 1

```
$ printf 'food\n' | python3 -m budget_app category remove --data-dir demo
삭제할 카테고리명: [오류] 'food' 카테고리를 사용 중인 거래가
있어 삭제할 수 없습니다.
[힌트] 해당 카테고리를 사용하는 거래를 먼저 다른 카테고리로
옮기거나 삭제하세요.
$ echo $?
1
```

사용하지 않는 hobby → 삭제

```
$ printf 'hobby\n' | python3 -m budget_app category remove --data-dir
demo
삭제할 카테고리명: [삭제 완료] category=hobby
```

항목 1-4 | budget set 저장과 예산 사용률·초과 경고

Q. budget set이 저장되며, summary에서 예산 사용률/초과 여부가 출력되는가?

A. budget set은 budgets.jsonl에 월별로 저장(같은 월은 덮어쓰)하고, summary가 지출÷예산으로 사용률을 계산해 출력합니다. 지출이 예산보다 크면 [경고]를 추가로 출력합니다.

코드 근거 (budget_app/파일:행)

repository.py:167-171

```
167     def set(self, month: str, amount: int) -> None:
168         budgets = self._all()
169         budgets[month] = amount
170         rows = [{"month": m, "amount": a} for m, a in
sorted(budgets.items())]
171         _atomic_write_jsonl(self.path, rows)
```

service.py:241-246

```
241         budget = self.budget_store.get(month)
242         usage_percent = None
243         over_budget = False
244         if budget is not None and budget > 0:
245             usage_percent = round(expense / budget * 100, 1)
246             over_budget = expense > budget
```

cli.py:119-122

```
119     if result["budget"] is not None:
120         print(f"예산: {result['budget']}원 (사용률
{result['usage_percent']}%)")
121     if result["over_budget"]:
122         print("[경고] 이번 달 예산을 초과했습니다.")
```

실행 근거 (b2-1 폴더에서 실행)

```
$ python3 -m budget_app budget set --month 2024-01 --amount 10000 --data-
dir demo
[저장 완료] 2024-01 예산 10000원
$ cat demo/budgets.jsonl
{"month": "2024-01", "amount": 10000}
```

```
$ python3 -m budget_app summary --month 2024-01 --data-dir demo
총 수입: 50000원
총 지출: 19250원
잔액: 30750원
예산: 10000원 (사용률 192.5%)
[경고] 이번 달 예산을 초과했습니다.
```

```
지출 TOP 5
1) food 18000원
2) transport 1250원
```

항목 1-5 | import/export CSV 스키마 (UTF-8 · 헤더 · 컬럼)

Q. import/export가 명시된 CSV 스키마(UTF-8, 헤더, 컬럼)로 동작하는가?

A. 컬럼은 CSV_FIELDS 한 곳에 정의(date,type,category,amount,memo,tags)하고, import는 UTF-8로 읽어 헤더 이름으로 값을 꺼내며(DictReader), export는 같은 컬럼 순서로 헤더를 먼저 씁니다(DictWriter).

코드 근거 (budget_app/파일:행)

service.py:18

```
18 CSV_FIELDS = ["date", "type", "category", "amount", "memo", "tags"]
```

service.py:268-270

```
268         with csv_path.open("r", encoding="utf-8", newline="") as f:
269             reader = csv.DictReader(f)
270             for row in reader:
```

service.py:310-312

```
310         with out.open("w", encoding="utf-8", newline="") as f:
311             writer = csv.DictWriter(f, fieldnames=CSV_FIELDS)
312             writer.writeheader()
```

service.py:321

```
321                 "tags": ",".join(tx.tags),
```

실행 근거 (b2-1 폴더에서 실행)

가져온 파일

```
$ cat imp.csv
date,type,category,amount,memo,tags
2024-01-20,expense,transport,1250,지하철,
2024-01-21,expense,없는카테고리,1000,깨짐,
2024-01-22,income,salary,50000,보너스,
```

내보낸 파일 (항목 1-1의 export 결과, 헤더 포함)

```
$ cat out.csv
date,type,category,amount,memo,tags
2024-01-15,expense,food,18000,점심,meal
2024-01-25,income,salary,3000000,월급,
2024-01-20,expense,transport,1250,지하철,
2024-01-22,income,salary,50000,보너스,
```

항목 1-6·7 | 오류 메시지 · 해결 힌트 · 종료 코드

Q. 잘못된 입력/파일 오류에서 스택트레이스 없이 오류 메시지와 해결 힌트를 출력하는가?

Q. 오류 상황에서 종료 코드가 0이 아님을 확인할 수 있는가?

A. 예상한 오류는 `AppError`(메시지+힌트)로 올리고, 모든 명령을 감싼 `handle_errors`가 이를 잡아 표준에러에 [오류]/[힌트]만 출력하고 1을 반환합니다. `__main__`이 이 값을 `sys.exit`로 넘깁니다.

코드 근거 (budget_app/파일:행)

decorators.py:26-34

```
26     except AppError as e:
27         print(f"[오류] {e.message}", file=sys.stderr)
28         if e.hint:
29             print(f"[힌트] {e.hint}", file=sys.stderr)
30         return 1
31     except Exception as e: # 예상 못한 오류도 스택트레이스 없이 처리
32         print(f"[오류] 처리 중 문제가 발생했습니다: {e}",
file=sys.stderr)
33         print("[힌트] 입력값을 확인하거나, 문제가 반복되면 관리자에게 문의하세
요.", file=sys.stderr)
34         return 1
```

__main__.py:6-7

```
6 if __name__ == "__main__":
7     sys.exit(main())
```

실행 근거 (b2-1 폴더에서 실행)

```
$ python3 -m budget_app update --id TX-999999 --amount 1000 --data-dir
demo
[오류] 존재하지 않는 거래 id입니다: TX-999999
[힌트] `list` 명령으로 올바른 id를 확인하세요.
$ echo $?
1
```

```
$ python3 -m budget_app export --out x.csv --data-dir demo
[오류] export는 --month 또는 --from/--to 조건이 최소 1개 필요합니다.
[힌트] 예: export --out export.csv --month 2024-01
$ echo $?
1
```

```
$ python3 -m budget_app import --from nofile.csv --data-dir demo
[오류] 파일을 찾을 수 없습니다: nofile.csv
$ echo $?
1
$ python3 -m budget_app summary --month 2024-13 --data-dir demo
[오류] 월 형식이 올바르지 않습니다 (YYYY-MM).
[힌트] 예: 2024-01
$ echo $?
1
```

항목 2-1 | 모듈 분리와 책임

Q. 코드가 3개 이상 모듈로 분리되어 있고, 각 모듈의 책임을 “어떻게” 나눴는지 설명할 수 있는가?

A. 바뀌는 이유가 다른 것끼리 나눴습니다: 화면(cli) · 규칙(service) · 파일(repository) · 데이터(models)
· 공통 처리(decorators). import 방향이 cli → service → repository → models 한쪽으로만 흐릅니다.

코드 근거 (budget_app/파일:행)

cli.py:1-3

```
1 """CLI 계층: 인자 파싱, 대화형 입력, 화면 출력만 담당한다.
2
3 실제 검증/저장 로직은 service.py / repository.py에 위임한다.
```

service.py:1-4

```
1 """비즈니스 로직(검증 + 규칙)을 담당하는 서비스 계층.
2
3 CLI는 사용자 입력을 받아 이 계층의 메서드만 호출하고,
4 파일을 어떻게 읽고 쓰는지 repository 계층에 위임한다.
```

repository.py:1-6

```
1 """파일 입출력(저장소) 계층.
2
3 - JSONL(줄 단위 JSON) 형식으로 transactions / categories / budgets 3개 파
일에
4 각각 영구 저장한다.
5 - 조회는 제너레이터(yield)로 한 줄씩 읽어 스트리밍 처리한다.
6 - 수정/삭제는 임시 파일에 새 내용을 쓴 뒤 os.replace로 원자적으로 교체한다.
```

models.py:1

```
1 """데이터 모델 정의 (Transaction 등)."""
```

실행 근거 (b2-1 폴더에서 실행)

```
$ ls budget_app/*.py
__init__.py __main__.py cli.py decorators.py
errors.py models.py repository.py service.py
```

의존 방향 확인 (repository는 service/cli를 import하지 않음)

```
$ grep -n "^from budget_app" budget_app/*.py
__main__.py:4: from budget_app.cli import main
repository.py:16:from budget_app.models import Transaction
decorators.py:11:from budget_app.errors import AppError
service.py:13:  from budget_app.errors import AppError
service.py:14:  from budget_app.models import ...
service.py:15:  from budget_app.repository import ...
cli.py:10:    from budget_app.decorators import ...
cli.py:11:    from budget_app.errors import AppError
cli.py:12:    from budget_app.repository import ...
cli.py:13:    from budget_app.service import BudgetService
```

항목 2-2 | 클래스 책임 경계

Q. 최소 2개 이상의 클래스에 부여한 책임 경계를 “어떻게” 정했는지 설명할 수 있는가?

A. Transaction=거래 한 건의 데이터, Repository/Store=파일 하나씩 입출력, BudgetService=검증과 규칙.
Service는 저장소를 생성자로 받아 쓰기만 하고(파일 형식을 모름), 저장소는 업무 규칙을 모릅니다.

코드 근거 (budget_app/파일:행)

models.py:9-19

```
9 @dataclass
10 class Transaction:
11     """거래 내역 1건을 표현하는 데이터 모델."""
12
13     id: str
14     date: str # YYYY-MM-DD
15     type: str # income | expense
16     category: str
17     amount: int # 양수
18     memo: str = ""
19     tags: list[str] = field(default_factory=list)
```

repository.py:46-47, 122-123, 155-156

```
46 class TransactionRepository:
47     """transactions.jsonl 파일 하나를 담당하는 저장소."""
48
49
122 class CategoryStore:
123     """categories.jsonl 파일 하나를 담당하는 저장소."""
124
125
155 class BudgetStore:
156     """budgets.jsonl 파일 하나를 담당하는 저장소 (월별 예산 금액)."""
```

실행 근거 (b2-1 폴더에서 실행)

```
$ grep -n "^class" budget_app/*.py
errors.py:5: class AppError(Exception):
service.py:68: class BudgetService:
models.py:10: class Transaction:
repository.py:46: class TransactionRepository:
repository.py:122: class CategoryStore:
repository.py:155: class BudgetStore:
```

한 기능이 세 클래스에 어떻게 나뉘는가

```
경계 예시: 카테고리 삭제
규칙 판단 → BudgetService.remove_category (service.py:88)
사용 여부 → TransactionRepository.used_categories
              (repository.py:118)
파일 수정 → CategoryStore.remove (repository.py:146)
```

코드 근거 (budget_app/파일:행)

service.py:68-74

```
68 class BudgetService:
69     def __init__(
70         self,
71         tx_repo: TransactionRepository,
72         cat_store: CategoryStore,
73         budget_store: BudgetStore,
74     ) -> None:
```

항목 2-3 | 파일 기반 update/delete의 안전한 처리

Q. 파일 기반 update/delete를 “어떻게” 안전하게 처리했는지 설명할 수 있는가?

A. JSONL은 중간 행만 덮어쓸 수 없어 전체를 다시 씁니다. 원본을 직접 쓰지 않고 .tmp 파일에 먼저 쓴 뒤 os.replace로 한 번에 교체하므로, 쓰는 도중 중단돼도 원본이 반쯤 쓰인 상태로 남지 않습니다.

코드 근거 (budget_app/파일:행)

repository.py:30-37

```
30 def _atomic_write_jsonl(path: Path, rows: Iterable[dict]) -> None:
31     """임시 파일에 먼저 쓰고 os.replace로 교체해 쓰다가 중단돼도 원본이 깨지지 않게
32     한다."""
33     path.parent.mkdir(parents=True, exist_ok=True)
34     tmp_path = path.with_suffix(path.suffix + ".tmp")
35     with tmp_path.open("w", encoding="utf-8") as f:
36         for row in rows:
37             f.write(json.dumps(row, ensure_ascii=False) + "\n")
38     os.replace(tmp_path, path)
```

repository.py:94, 97-104

```
94 def update(self, tx_id: str, apply_changes:
95     Callable[[Transaction], Transaction]) -> bool:
96     :
97     for tx in self.iter_all():
98         if tx.id == tx_id:
99             tx = apply_changes(tx)
100             found = True
101             rows.append(tx.to_dict())
102     if found:
103         _atomic_write_jsonl(self.path, rows)
104     return found
```

실행 근거 (b2-1 폴더에서 실행)

교체 후 .tmp 파일이 남지 않음

```
$ python3 -m budget_app delete --id TX-000003 --data-dir demo
[삭제 완료] id=TX-000003
$ ls demo
budgets.jsonl categories.jsonl transactions.jsonl
```

없는 id → found=False라 파일을 다시 쓰지 않음 (repository.py:114)

```
$ python3 -m budget_app delete --id TX-999999 --data-dir demo
[오류] 존재하지 않는 거래 id입니다: TX-999999
[힌트] `list` 명령으로 올바른 id를 확인하세요.
```

한계: 전원 차단 등 저장장치 장애까지 보장하지는 않음
(fsync 미사용, 동시 실행 잠금 없음)

항목 3-1 | list/search 제너레이터 스트리밍

Q. list/search를 제너레이터로 스트리밍 처리한 방식을 “어떻게” 구현했고, “왜” 유리한지 설명할 수 있는가?

A. 파일을 한 줄씩 읽어 yield하고, list는 deque(maxlen=N)로 최근 N건만 유지합니다. 거래가 늘어도 메모리가 N건 분량으로 고정됩니다. 단, search는 최신순 정렬 때문에 일치한 결과는 리스트로 모읍니다.

코드 근거 (budget_app/파일:행)

repository.py:19-27

```
19 def _read_jsonl(path: Path) -> Iterator[dict]:
20     """파일이 없으면 빈 이터레이터, 있으면 한 줄씩 JSON으로 파싱해 yield."""
21     if not path.exists():
22         return
23     with path.open("r", encoding="utf-8") as f:
24         for line in f:
25             line = line.strip()
26             if line:
27                 yield json.loads(line)
```

repository.py:52-55

```
52 def iter_all(self) -> Iterator[Transaction]:
53     """파일을 한 줄씩 읽어 Transaction으로 변환해 yield (스트리밍)."""
54     for row in _read_jsonl(self.path):
55         yield Transaction.from_dict(row)
```

repository.py:79-82

```
79 window: deque[Transaction] = deque(maxlen=limit)
80 for tx in self.iter_all():
81     window.append(tx)
82 yield from reversed(window)
```

실행 근거 (b2-1 폴더에서 실행)

iter_all()은 제너레이터 — next()로 한 건씩 꺼냄

```
$ python3 -c "from pathlib import Path; from budget_app.repository import
TransactionRepository as R; g =
R(Path('demo/transactions.jsonl')).iter_all(); print(type(g).__name__);
print(next(g))"
generator
Transaction(id='TX-000001', date='2024-01-15', type='expense',
category='food', amount=18000, memo='점심', tags=['meal'])
```

```
$ python3 -m budget_app list --limit 1 --data-dir demo
TX-000004 | 2024-01-22 | income | salary | 50000 | 보너스
```

왜 유리한가: 실측 비교 (항목 4-2 측정과 같은 데이터)

거래 10만 건 파일(12.9MB)에서 최대 메모리 (/usr/bin/time -l)	
list --limit 5	13 MB (스트리밍)
search --category food	23 MB (일치한 약 2만 건을 모음)
update	79 MB (전체 행을 리스트로 모음)

코드 근거 (budget_app/파일:행)

repository.py:91-92

```
91 matched = [tx for tx in self.iter_all() if predicate(tx)]
92 yield from reversed(matched)
```

항목 3-2 | 데코레이터로 분리한 공통 기능

Q. 데코레이터로 분리한 공통 기능이 무엇이며, “왜” 분리가 필요했는지 설명할 수 있는가?

A. handle_errors(오류→메시지·힌트·종료 코드)와 log_execution(실행 로그·시간 측정)을 모든 cmd_* 함수에 붙였습니다. 12개 명령에 같은 try/except와 로그 코드를 반복하지 않고, 명령 함수는 기능만 담습니다.

코드 근거 (budget_app/파일:행)

decorators.py:39-58

```
39 def log_execution(func: Callable) -> Callable:
40     """명령 실행 시작/종료와 걸린 시간을 budget_app.log 파일에 기록한다."""
41
42     @functools.wraps(func)
43     def wrapper(*args, **kwargs):
44         start = time.perf_counter()
45         timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
46         status = "OK"
47         try:
48             return func(*args, **kwargs)
49         except Exception:
50             status = "ERROR"
51             raise
52         finally:
53             elapsed_ms = (time.perf_counter() - start) * 1000
54             with LOG_FILE.open("a", encoding="utf-8") as f:
55                 f.write(
56                     f"[{timestamp}] {func.__name__} status={status}\n"
57
58                 f"elapsed={elapsed_ms:.1f}ms\n"
```

실행 근거 (b2-1 폴더에서 실행)

적용된 명령 함수 수

```
$ grep -c "^@handle_errors" budget_app/cli.py
12
```

log_execution 기록 (성공·실패 모두 남음, 시각은 실행 때마다 다름)

```
$ tail -3 budget_app.log
[2026-09-27 15:23:36] cmd_delete status=OK elapsed=0.7ms
[2026-09-27 15:23:36] cmd_delete status=ERROR elapsed=0.3ms
[2026-09-27 15:23:36] cmd_list status=OK elapsed=0.2ms
```

functools.wraps로 감싼 뒤에도 원래 이름 유지 → 로그에 명령명 기록

```
$ python3 -c "from budget_app.cli import cmd_add;
print(cmd_add.__name__)"
cmd_add
```

코드 근거 (budget_app/파일:행)

cli.py:41-43

```
41 @handle_errors
42 @log_execution
43 def cmd_add(args: argparse.Namespace) -> int:
```

항목 3-3 | 타입 힌트의 이점

Q. 타입 힌트를 적용해 얻는 이점을 실제 코드 예로 “어떻게” 확인했고 “왜” 도움이 되는지 설명할 수 있는가?

A. 함수 정의만 보고 입력·반환 타입을 알 수 있고(str→int, 생략 가능한 str | None, 제너레이터 반환), IDE 자동완성과 정적 분석에 쓰입니다. 단, 실행 중 값 검사는 하지 않으므로 검증은 validate_*가 합니다.

코드 근거 (budget_app/파일:행)

service.py:52-59

```
52 def validate_amount(amount_str: str) -> int:
53     try:
54         amount = int(amount_str)
55     except ValueError:
56         raise AppError("금액은 숫자여야 합니다.", hint="예: 15000")
57     if amount <= 0:
58         raise AppError("금액은 0보다 큰 양수여야 합니다.", hint="예: 15000")
59     return amount
```

service.py:134-142

```
134 def search_transactions(
135     self,
136     date_from: str | None = None,
137     date_to: str | None = None,
138     category: str | None = None,
139     type_: str | None = None,
140     query: str | None = None,
141     tag: str | None = None,
142 ) -> Iterator[Transaction]:
```

실행 근거 (b2-1 폴더에서 실행)

코드에 적은 타입 정보를 실행 환경에서 그대로 읽을 수 있음

```
$ python3 -c "import typing; from budget_app.service import
validate_amount, BudgetService;
print(typing.get_type_hints(validate_amount));
print(typing.get_type_hints(BudgetService.search_transactions)
['date_from'])"
{'amount_str': <class 'str'>, 'return': <class 'int'>}
str | None
```

잘못된 값은 타입 힌트가 아니라 validate_amount(service.py:53-56)가 막음

```
$ python3 -m budget_app add --data-dir demo
날짜(YYYY-MM-DD): 2024-01-15
타입(income/expense): expense
카테고리: food
금액(양수): abc
[오류] 금액은 숫자여야 합니다.
[힌트] 예: 15000
금액(양수):                ← 다시 입력 요청
```

코드 근거 (budget_app/파일:행)

repository.py:52

```
52 def iter_all(self) -> Iterator[Transaction]:
```

mypy 같은 정적 분석 도구는 이 환경에 설치되어 있지 않아 실행하지 않았습니다.

항목 4-1 | 저장 포맷: JSONL vs CSV

Q. JSONL과 CSV 중 선택한 저장 포맷의 장단점을 비교하고, “왜” 그 포맷을 택했는지 근거를 말할 수 있는가?

A. JSONL: 한 줄 = 한 거래라 추가는 끝에 한 줄 쓰기, tags 같은 목록도 그대로 저장 / 단점: 검색·수정은 전체 처리.
CSV: 스프레드시트 호환이 장점이나 목록은 "a,b" 문자열로 바뀌어야 함 → 내부 저장은 JSONL, 교환은 CSV.

코드 근거 (budget_app/파일:행)

repository.py:40-43

```
40 def _append_jsonl(path: Path, row: dict) -> None:
41     path.parent.mkdir(parents=True, exist_ok=True)
42     with path.open("a", encoding="utf-8") as f:
43         f.write(json.dumps(row, ensure_ascii=False) + "\n")
```

models.py:19

```
19     tags: list[str] = field(default_factory=list)
```

service.py:321

```
321         "tags": ",".join(tx.tags),
```

service.py:62-65

```
62 def parse_tags(tags_str: str) -> list[str]:
63     if not tags_str:
64         return []
65     return [t.strip() for t in tags_str.split(",") if t.strip()]
```

실행 근거 (b2-1 폴더에서 실행)

JSONL: 숫자·목록 타입이 그대로 보존됨

```
$ head -1 demo/transactions.jsonl
{"id": "TX-000001", "date": "2024-01-15", "type": "expense",
 "category": "food", "amount": 18000, "memo": "점심",
 "tags": ["meal"]}
```

CSV: 모든 값이 문자열, tags는 쉼표로 이어 붙인 문자열

```
$ sed -n 2p out.csv
2024-01-15,expense,food,18000,점심,meal
```

판단 근거

추가(add)가 가장 잦은 작업 → JSONL은 append 한 번
표준 라이브러리 json만으로 읽기/쓰기
사용자 교환은 CSV로 따로 제공 (import/export)

항목 4-2 | 거래 10만 건에서의 병목과 개선

Q. 거래가 10만 건으로 늘어난다면, 현재 구조에서 병목이 어디이며 “어떻게” 개선할지 설명할 수 있는가?

A. 실측 결과 가장 큰 병목은 import입니다. 행마다 next_id()가 파일 전체를 다시 읽어 1,000행에 515초가 걸렸습니다. 개선: ID 카운터 저장, import 일괄 쓰기, 날짜·카테고리 인덱스, 규모가 더 크면 SQLite.

코드 근거 (budget_app/파일:행)

repository.py:57-65

```
57     def next_id(self) -> str:
58         max_no = 0
59         for tx in self.iter_all():
60             try:
61                 no = int(tx.id.split("-")[-1])
62                 max_no = max(max_no, no)
63             except ValueError:
64                 continue
65         return f"TX-{max_no + 1:06d}"
```

service.py:118-120, 128

```
118         self._validate_category_exists(category)
119         tx = Transaction(
120             id=self.tx_repo.next_id(),
121             :
128         self.tx_repo.add(tx)
```

실행 근거 (b2-1 폴더에서 실행)

\$ /usr/bin/time -l python3 -m budget_app <명령> --data-dir bench (거래 10만 건, 12.9MB)

command	time	max memory
list --limit 5	0.52 s	13 MB
search --category food	0.57 s	23 MB
summary --month 2024-03	0.54 s	13 MB
export --month 2024-03	0.56 s	17 MB
add (1 row)	0.59 s	13 MB
update (1 row)	1.46 s	79 MB
import (1,000 rows)	515.46 s	13 MB

병목 원인과 개선

- ① import: 행마다 next_id()로 전체 읽기
→ 비용 = 행 수 × 거래 수
→ 마지막 id를 따로 저장하고 한 번에 append
- ② 조회: 인덱스 없이 전체 순회 → 월별 파일 분할이나 인덱스
- ③ update/delete: 전체를 메모리에 모아 재작성
→ 한 줄씩 .tmp로 흘려 쓰기, 또는 SQLite로 이전

항목 4-3 | 깨진 CSV 행과 사용자 신뢰

Q. import CSV에 일부 깨진 행이 섞이면, “어떻게” 처리해 사용자 신뢰를 지킬지(부분 성공/롤백/리포트) 설명할 수 있는가?

A. 현재는 행 단위 부분 성공입니다. 행마다 검증해 실패(AppError·열 누락)한 행만 건너뛰고 개수를 알려줍니다.
한계: 몇 번째 행이 왜 실패했는지 알려주지 않음 → 행 번호·사유 리포트, 필요하면 전체 검증 후 일괄 저장(롤백).

코드 근거 (budget_app/파일:행)

service.py:266-283

```
266         imported = 0
267         skipped = 0
268         with csv_path.open("r", encoding="utf-8", newline="") as f:
269             reader = csv.DictReader(f)
270             for row in reader:
271                 try:
272                     self.add_transaction(
273                         date=row["date"],
274                         type=row["type"],
275                         category=row["category"],
276                         amount=row["amount"],
277                         memo=row.get("memo", "") or "",
278                         tags=row.get("tags", "") or "",
279                     )
280                     imported += 1
281                 except (AppError, KeyError):
282                     skipped += 1
283             return imported, skipped
```

실행 근거 (b2-1 폴더에서 실행)

정상 행 2개는 저장, 깨진 행 1개는 건너뛴 (행 번호·사유는 출력 안 함)

```
$ cat imp.csv
date,type,category,amount,memo,tags
2024-01-20,expense,transport,1250,지하철,
2024-01-21,expense,없는카테고리,1000,깨짐, ← 미등록 카테고리
2024-01-22,income,salary,50000,보너스,
$ python3 -m budget_app import --from imp.csv --data-dir demo
[완료] imported=2, skipped=1
$ echo $?
0
```

개선안

리포트: [건너뛴] 3행 category=없는카테고리 (미등록 카테고리)

롤백: 모든 행을 먼저 검증 → 하나라도 실패하면 저장 안 함,
전부 통과하면 한 번에 append

선택: --strict 옵션으로 롤백/부분 성공을 사용자가 고를

마무리 | 발표용 요약 답변

거래·카테고리·예산을 JSONL 파일 세 개에 나누어 저장하고 (cli.py:16-21), CLI·서비스·저장소·모델로 책임을 분리했습니다.

파일은 제너레이터로 한 줄씩 읽고 (repository.py:19-27), 목록은 최근 N건만 유지합니다 (repository.py:79-82). 수정과 삭제는 임시 파일에 먼저 기록한 뒤 교체합니다 (repository.py:30-37).

공통 오류 처리와 실행 로그는 데코레이터로 적용했고 (decorators.py), 타입 힌트로 입출력 계약을 드러냈습니다.

한계도 확인했습니다. 검색 결과는 메모리에 모이고, CSV 가져오기는 행 단위 부분 성공이며 실패 사유를 알려주지 않습니다. 10만 건 실측에서 import 1,000행이 515초 걸렸으므로 ID 생성과 일괄 쓰기부터 개선하겠습니다.